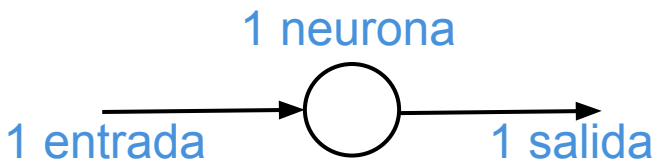
Red Neuronal

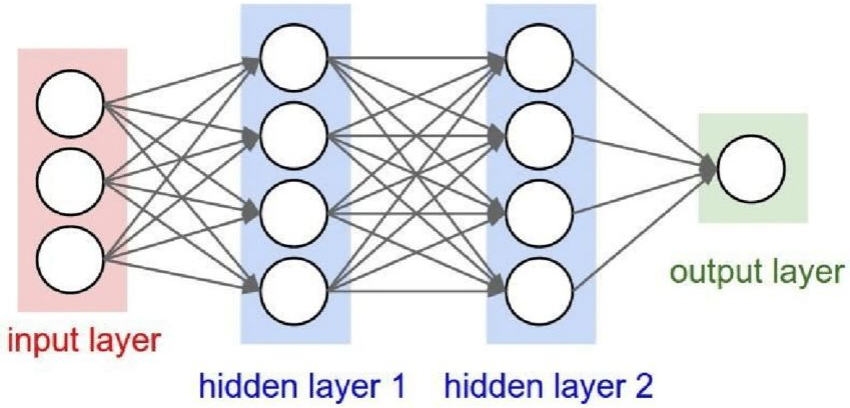
Red Neuronal y codificacion

Neurona simple

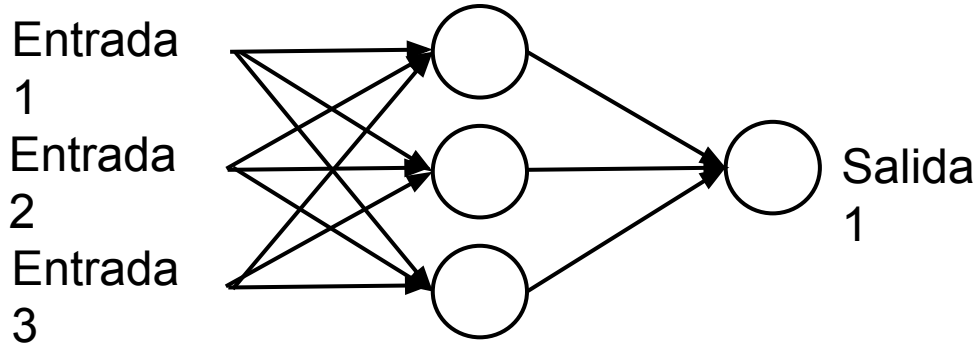
```
modelo_simple = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])  # solo una neurona
])
```



Red Neuronal



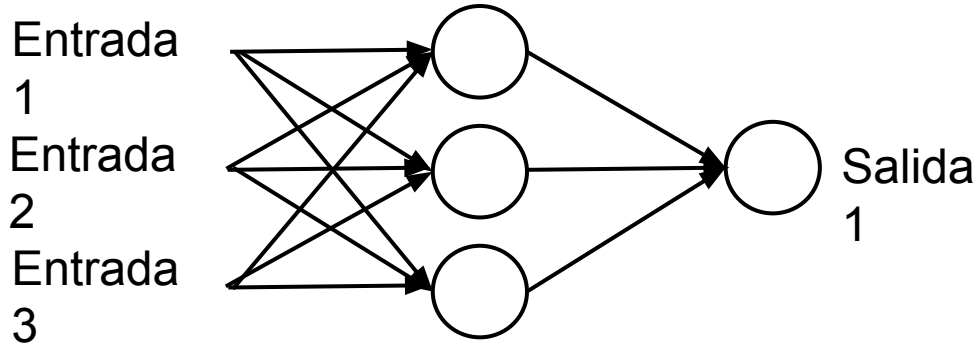
Red neuronal simple



- 3 capas
  - **Capa de entrada: n variables (depende de la cantidad de columnas, supongamos 3 columnas)**
    - **input_shape=[X.shape[0]] #cantidad de filas**
    - **input_shape=[X.shape[1]] #cantidad de columnas**
      - Cantidad de entradas: X.shape #dataframe: X, shape: dimensión
        - Si X tiene 4 columnas: input_shape =4
  - Capa oculta
    - 3 neuronas
  - Capa de salida: 1 neurona

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=3, input_shape=[X.shape[1]]),
    tf.keras.layers.Dense(units=1)
])
```

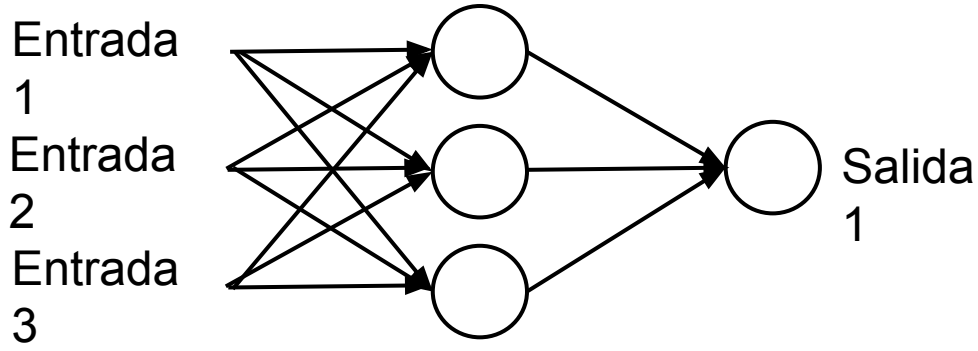
Red neuronal simple



- 3 capas
  - Capa de entrada: n variables (depende de la cantidad de columnas, supongamos 3 columnas)
    - input_shape=[X.shape[0] #cantidad de filas
    - input_shape=[X.shape[1] #cantidad de columnas
      - Cantidad de entradas: X.shape #dataframe: X, shape: dimensión
        - Si X tiene 4 columnas: input_shape =[4]
  - **Capa oculta**
    - **3 neuronas**
  - Capa de salida: 1 neurona

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=3, input_shape=[X.shape[1]]),
    tf.keras.layers.Dense(units=1)
])
```

Red neuronal simple



- 3 capas
  - Capa de entrada: n variables (depende de la cantidad de columnas, supongamos 3 columnas)
    - input_shape=[X.shape[0] #cantidad de filas
    - input_shape=[X.shape[1] #cantidad de columnas
  - Capa oculta
    - 3 neuronas
    - Cantidad de entradas: X.shape #dataframe: X, shape: dimensión
      - Si X tiene 4 columnas: input_shape =[4]
  - **Capa de salida: 1 neurona**

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=3, input_shape=[X.shape[1]]),
    tf.keras.layers.Dense(units=1)
])
```

Conversión de valores texto a numérico

- Las redes neuronales trabajan con números debido a que internamente son un modelo matemático, no con texto.

  - Variables categóricas
    - `"Soleado"`, `"Nublado"`, `"Lluvioso"`
    - `"Alto"`, `"Medio"`, `"Bajo"`
    - `"Masculino"`, `"Femenino"`
  - Texto libre
    - `"La película estuvo muy buena"`
    - `"El clima hoy es terrible"`

Conversión de Texto a números

Técnica	Cómo funciona
Bag of Words	Cuenta cuántas veces aparece cada palabra
TF-IDF	Similar pero penaliza palabras muy comunes
Word2Vec	Convierte cada palabra en un vector numérico con significado semántico
Embeddings (BERT, GPT)	Representaciones profundas que entienden contexto
LLMs	Tokenizar, Codificar tokens, Embedding (vectorizar), Positional (posición token)

Codificación one-hot

- La codificación One-Hot (One-Hot Encoding) es una técnica para convertir datos categóricos (texto o categorías) en números en un dataset/dataframe.

one-hot encoding

	Temperatura	Tipo_dia	Consumo
0	20.0	Soleado	400.0
1	25.0	Soleado	420.0
2	30.0	Nublado	500.0
3	35.0	Lluvioso	550.0

```
data = pd.get_dummies(data, columns=['Tipo_dia'], dtype=int)
```

	Temperatura	Consumo	Tipo_dia_Lluvioso	Tipo_dia_Nublado	Tipo_dia_Soleado
0	20.0	400.0	0	0	1
1	25.0	420.0	0	0	1
2	30.0	500.0	0	1	0
3	35.0	550.0	1	0	0

Ejemplo

- Tiene tres categorías para color: rojo, azul y verde.
- Solución en one hot encoding:

Rojo	Azul	Verde
1	0	0
0	1	0
0	0	1

No es recomendable asignar números

Por ejemplo:

Rojo = 1

Azul = 2

Verde = 3

El modelo podría creer que: (como si hubiese un orden matemático)

Rojo > Azul > Verde

Integer encoding
Labelencoder

	Temperatura	Tipo_dia	Consumo
0	20.0	Soleado	400.0
1	25.0	Soleado	420.0
2	30.0	Nublado	500.0
3	35.0	Lluvioso	550.0

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
data['Tipo_dia'] = le.fit_transform(data['Tipo_dia'])
print(data)
```

	Temperatura	Tipo_dia	Consumo
0	20.0	2	400.0
1	25.0	2	420.0
2	30.0	1	500.0
3	35.0	0	550.0

Ejemplo

Predicción del precio de una habitación de hotel

Una pequeña empresa desea predecir el precio de una habitación de hotel según tres factores:

- Temporada (Alta, Media, Baja)
- Número de noches reservadas
- Cantidad de personas

Datos de entrenamiento:

Temporada	Noches	Personas	Precio (QTZ)
Alta	2	2	180
Media	2	2	130
Baja	1	1	70
Alta	5	4	420
Media	4	3	260

Tareas a realizar:

- Aplicar **one-hot encoding** a “Temporada”.
- Normalizar todas las variables con MinMaxScaler.
- Entrenar una red neuronal con 3 capas:
  - Capa de entrada: 5 entradas #despues de aplicar one-hot enconding
  - Capa oculta con 3 neuronas.
  - Capa de salida con 1 neurona (predicción del precio).

Ejercicio

Predicción del nivel de estrés de un estudiante (0-100)

- Horas de sueño, Minutos de ejercicio diario, Tazas de café consumidas
- Cursos asignados, proyectos pendientes, horas de estudio y tipo de fin semana en la que se encuentra.

Datos de entrenamiento:

	horas_sueno	minutos_ejercicio	tazas_cafe	cantidad_cursos	proyectos_pendientes	horas_estudio	tipo_semana	nivel_estres
0	4	10.0	5	6	3	5.0	finales	95
1	7	30.0	2	4	1	2.0	normal	40
2	5	15.0	4	7	4	6.0	parciales	88
3	6	20.0	3	5	2	NaN	parciales	65
4	8	40.0	1	3	0	1.0	vacaciones	15
5	3	0.0	6	8	5	7.0	finales	99
6	9	60.0	0	2	0	1.0	vacaciones	5
7	5	12.0	5	6	3	5.0	parciales	82
8	6	25.0	3	5	2	4.0	normal	60
9	7	35.0	2	4	1	3.0	normal	35
10	4	8.0	4	7	4	6.0	finales	90
11	8	50.0	1	3	0	2.0	NaN	12
12	5	18.0	4	6	3	5.0	parciales	80
13	6	22.0	3	5	2	4.0	normal	55
14	7	28.0	2	4	1	3.0	normal	38
15	3	5.0	6	8	5	7.0	finales	97
16	9	70.0	0	2	0	1.0	vacaciones	8
17	4	NaN	5	7	4	6.0	parciales	85
18	6	24.0	3	5	2	4.0	normal	58
19	5	16.0	4	6	3	5.0	parciales	75

Tareas:

- Normalizar el training data.
- Entrenar la red neuronal
- Solicitar horas de sueño, minutos de ejercicio y tazas de café, y predecir el nivel de estrés.
- Interpretar el resultado: cambiar la cantidad de neuronas aumentando o disminuyendo la cantidad de neuronas o capas. Posteriormente observe si las predicciones del modelo mejoran o no.